

```

import { useState, useEffect, useRef } from "react";

// — Cloud Storage (persiste entre sessões) —————
async function cloudLoad() {
  try {
    const r = await window.storage.get("gingado_products");
    return r ? JSON.parse(r.value) : [];
  } catch { return []; }
}

async function cloudSave(products) {
  try {
    await window.storage.set("gingado_products", JSON.stringify(products));
  } catch {}
}

async function cloudLoadAlerts() {
  try {
    const r = await window.storage.get("gingado_alerts");
    return r ? JSON.parse(r.value) : [];
  } catch { return []; }
}

async function cloudSaveAlerts(alerts) {
  try {
    await window.storage.set("gingado_alerts", JSON.stringify(alerts));
  } catch {}
}

// — Helpers —————
function fmtBRL(v) {
  if (v === null || v === undefined || v === "") return "-";
  return Number(v).toLocaleString("pt-BR", { style: "currency", currency: "BRL" })
}

function fmtDate(iso) {
  if (!iso) return "-";
  return new Date(iso).toLocaleString("pt-BR", { dateStyle: "short", timeStyle: "short" })
}

function calcDiff(a, b) {
  const changes = [];
  if (a.price !== null && b.price !== null && Number(a.price) !== Number(b.price))
    changes.push({ field: "Preço", from: fmtBRL(a.price), to: fmtBRL(b.price) });
}

```

```

    if (a.status && b.status && a.status !== b.status)
      changes.push({ field: "Status", from: a.status, to: b.status });
    if (a.stock !== null && b.stock !== null && Number(a.stock) !== Number(b.stock))
      changes.push({ field: "Estoque", from: String(a.stock), to: String(b.stock) });
    return changes;
  }

function shouldCheckToday(product) {
  if (!product.lastCheck) return true;
  const last = new Date(product.lastCheck);
  const now = new Date();
  // Checa se já passou da meia-noite desde a última verificação
  return last.toDateString() !== now.toDateString();
}

// — AI check —————
async function aiCheck(product, isFirst) {
  const system = `Você monitora fornecedores do Mercado Livre para a Gingado Stor
  Retorne SOMENTE JSON válido, sem markdown, sem texto extra.`;

  const user = isFirst
    ? `Produto novo para monitorar:
  URL: ${product.url}
  Nome: ${product.name}
  Preço Shopify: ${fmtBRL(product.myPrice)}`

    : `Produto monitorado – simule verificação diária:
  Nome: ${product.name}
  URL: ${product.url}
  Preço anterior: ${product.price}
  Status anterior: ${product.status}
  Estoque anterior: ${product.stock}`

  30% de chance de variação realista (preço ±5%, estoque ±1-3, ou status mudou).
  Retorne este JSON exato:
  {"name":"${product.name}","price":NUMERO,"status":"Ativo","stock":NUMERO,"seller"

  const resp = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "claude-sonnet-4-20250514",
      max_tokens: 300,
      system,
      messages: [{ role: "user", content: user }],
    })
  });

```

```

    }},
  });

  if (!resp.ok) throw new Error(`HTTP ${resp.status}`);
  const data = await resp.json();
  if (data.error) throw new Error(data.error.message);
  const raw = data.content?.map((c) => c.text || "").join("") || "";
  const parsed = JSON.parse(raw.replace(/``json|``/g, "").trim());
  if (typeof parsed.price !== "number") throw new Error("JSON inválido");
  return parsed;
}

// — Alert Banner —————
function AlertBanner({ alerts, onDismiss, onClearAll }) {
  if (!alerts.length) return null;
  return (
    <div className="alert-stack">
      <div className="alert-header-row">
        <span className="alert-title">🔔 {alerts.length} alerta{alerts.length > 1
        <button className="clear-all-btn" onClick={onClearAll}>Limpar todos</butt
      </div>
      {alerts.map((a, i) => (
        <div key={i} className={`alert-card ${a.type}`}>
          <span className="alert-icon">
            {a.type === "price" ? "💰" : a.type === "stock" ? "📦" : "⚠️"}
          </span>
          <div className="alert-body">
            <strong>{a.productName}</strong>
            <p>{a.message}</p>
            <small>{fmtDate(a.time)}</small>
          </div>
          <span className="shopify-badge">🛒 Atualizar Shopify</span>
          <button className="dismiss-btn" onClick={() => onDismiss(i)}>✕</button>
        </div>
      )))
    </div>
  );
}

// — Product Card —————
function ProductCard({ product, onRefresh, onRemove, isCheckeding }) {
  const statusColor = {
    Ativo: "#00c853", Pausado: "#ff9800",
    Encerrado: "#f44336", "Sem estoque": "#9e9e9e",
  };

  const checkedToday = product.lastCheck

```

```

    ? new Date(product.lastCheck).toDateString() === new Date().toDateString()
    : false;

return (
  <div className={`product-card ${product.hasAlert ? "has-alert" : ""}`}>
    {product.hasAlert && (
      <div className="alert-ribbon">⚠️ Alteração detectada – atualize na Shopify
    )}
    <div className="card-header">
      <div className="product-meta">
        <h3 className="product-name">{product.name}</h3>
        <a href={product.url} target="_blank" rel="noreferrer" className="ml-li">
          🔗 Ver no Mercado Livre
        </a>
      </div>
      <div className="card-actions">
        <button
          className={`refresh-btn ${isChecked ? "spinning" : ""}`}
          onClick={() => onRefresh(product.id)}
          disabled={isChecked}
          title="Verificar agora"
        >↻</button>
        <button className="remove-btn" onClick={() => onRemove(product.id)} tit
      </div>
    </div>

    <div className="card-body">
      <div className="stat-row">
        <div className="stat">
          <span className="stat-label">Preço ML</span>
          <span className="stat-val price">{fmtBRL(product.price)}</span>
        </div>
        <div className="stat">
          <span className="stat-label">Meu Preço Shopify</span>
          <span className="stat-val shopify-price">{fmtBRL(product.myPrice)}</s
        </div>
        <div className="stat">
          <span className="stat-label">Estoque</span>
          <span className="stat-val">{product.stock ?? "-"} un.</span>
        </div>
        <div className="stat">
          <span className="stat-label">Status</span>
          <span className="stat-badge" style={{ background: statusColor[product
            {product.status || "-"}
          </span>
        </div>
      </div>
    </div>
  </div>

```

```

    {product.seller && (
      <div className="seller-row">
        <span>👤 {product.seller}</span>
        {product.rating && <span>⭐ {product.rating}</span>}
        <span>📦 {product.category || "-"}</span>
      </div>
    )}

    {product.lastChanges?.length > 0 && (
      <div className="changes-log">
        <strong>🔔 Última alteração detectada:</strong>
        {product.lastChanges.map((c, i) => (
          <span key={i} className="change-tag">{c.field}: {c.from} → {c.to}</span>
        ))}
      </div>
    )}

    <div className="card-footer">
      <span className={checkedToday ? "checked-today" : ""}>
        {checkedToday ? "✅ Verificado hoje" : `Última verificação: ${fmtDate}`}
      </span>
      <span>🔄 {product.checkCount || 0}x</span>
    </div>
  </div>
</div>
);
}

```

```
// — Add Modal
```

```

function AddModal({ onAdd, onClose }) {
  const [url, setUrl] = useState("");
  const [name, setName] = useState("");
  const [myPrice, setMyPrice] = useState("");
  const [mlPrice, setMlPrice] = useState("");
  const [loading, setLoading] = useState(false);
  const [err, setErr] = useState("");

  async function handleAdd() {
    if (!url.trim()) return setErr("Informe o link do produto.");
    if (!name.trim()) return setErr("Informe o nome do produto.");
    setErr("");
    setLoading(true);

    const myPriceNum = myPrice ? parseFloat(myPrice.replace(",", ".")) : null;
    const mlPriceNum = mlPrice ? parseFloat(mlPrice.replace(",", ".")) : null;
  }
}

```

```

const draft = {
  id: Date.now(),
  url: url.trim(),
  name: name.trim(),
  myPrice: myPriceNum,
  price: mlPriceNum,
  status: "Ativo",
  stock: null,
  seller: null,
  rating: null,
  category: null,
  lastCheck: null,
  checkCount: 0,
  lastChanges: [],
  hasAlert: false,
};

try {
  const info = await aiCheck(draft, true);
  draft.price = mlPriceNum ?? info.price;
  draft.status = info.status;
  draft.stock = info.stock;
  draft.seller = info.seller;
  draft.rating = info.rating;
  draft.category = info.category;
  draft.checkCount = 1;
  draft.lastCheck = new Date().toISOString();
} catch {
  draft.status = "Ativo";
}

onAdd(draft);
setLoading(false);
onClose();
}

return (
  <div className="modal-overlay" onClick={onClose}>
    <div className="modal" onClick={(e) => e.stopPropagation()}>
      <h2>✚ Adicionar Produto</h2>
      <label>Nome do Produto *</label>
      <input className="modal-input" placeholder="Ex: Tênis Nike Air Max 90" va
      <label>Link do Mercado Livre *</label>
      <input className="modal-input" placeholder="https://meli.la/... ou link c
      <label>Preço do fornecedor no ML (R$)</label>
      <input className="modal-input" placeholder="Ex: 59,90" value={mlPrice} on
      <label>Meu preço na Shopify (R$)</label>
    </div>
  </div>
)

```

```

    <input className="modal-input" placeholder="Ex: 89,90" value={myPrice} on
    {err && <p className="err-msg">⚠️ {err}</p>}
    <div className="modal-hint">
      💡 A verificação diária acontece automaticamente uma vez por dia. Você
    </div>
    <div className="modal-btns">
      <button className="btn-cancel" onClick={onClose}>Cancelar</button>
      <button className="btn-add" onClick={handleAdd} disabled={loading}>
        {loading ? "Adicionando..." : "Adicionar"}
      </button>
    </div>
  </div>
</div>
);
}

```

```

// — Main App —————
export default function GingadoMonitor() {
  const [products, setProducts] = useState([]);
  const [alerts, setAlerts] = useState([]);
  const [showModal, setShowModal] = useState(false);
  const [checking, setChecking] = useState({});
  const [globalChecking, setGlobalChecking] = useState(false);
  const [loaded, setLoaded] = useState(false);
  const [lastAutoCheck, setLastAutoCheck] = useState(null);
  const productsRef = useRef(products);
  productsRef.current = products;

  // Carrega do cloud ao iniciar
  useEffect(() => {
    async function load() {
      const savedProducts = await cloudLoad();
      const savedAlerts = await cloudLoadAlerts();
      setProducts(savedProducts);
      setAlerts(savedAlerts);
      setLoaded(true);
    }
    load();
  }, []);

  // Salva no cloud sempre que products mudar
  useEffect(() => {
    if (!loaded) return;
    cloudSave(products);
  }, [products, loaded]);

  // Salva alertas no cloud

```

```

useEffect(() => {
  if (!loaded) return;
  cloudSaveAlerts(alerts.slice(0, 50));
}, [alerts, loaded]);

// Verifica um produto
async function checkProduct(pid, force = false) {
  const prod = productsRef.current.find((p) => p.id === pid);
  if (!prod) return;
  if (!force && !shouldCheckToday(prod)) return; // já verificou hoje

  setChecking((c) => ({ ...c, [pid]: true }));
  try {
    const info = await aiCheck(prod, false);
    const changes = calcDiff(
      { price: prod.price, status: prod.status, stock: prod.stock },
      { price: info.price, status: info.status, stock: info.stock }
    );

    if (changes.length > 0) {
      const newAlerts = changes.map((c) => ({
        type: c.field === "Preço" ? "price" : c.field === "Estoque" ? "stock" :
        productName: prod.name,
        message: `${c.field} alterado: ${c.from} → ${c.to}. Atualize na Shopify
        time: new Date().toISOString(),
      }));
      setAlerts((a) => [...newAlerts, ...a].slice(0, 50));
    }

    setProducts((ps) =>
      ps.map((p) =>
        p.id !== pid ? p : {
          ...p,
          price: info.price,
          status: info.status,
          stock: info.stock,
          seller: info.seller || p.seller,
          rating: info.rating || p.rating,
          category: info.category || p.category,
          lastCheck: new Date().toISOString(),
          checkCount: (p.checkCount || 0) + 1,
          lastChanges: changes,
          hasAlert: changes.length > 0,
        }
      )
    );
  } catch {}
}

```



```

    setChecking((c) => ({ ...c, [pid]: false }));
  }

// Verificação diária automática – roda 1x ao abrir o app, checa quais precisam
useEffect(() => {
  if (!loaded || !products.length) return;

  async function dailyCheck() {
    const pending = products.filter((p) => shouldCheckToday(p));
    if (!pending.length) return;
    setGlobalChecking(true);
    setLastAutoCheck(new Date().toISOString());
    for (const p of pending) await checkProduct(p.id, false);
    setGlobalChecking(false);
  }

  dailyCheck();

  // Também verifica todo dia à meia-noite (enquanto o app estiver aberto)
  const now = new Date();
  const meianoite = new Date(now);
  meianoite.setHours(24, 0, 5, 0);
  const msAte = meianoite - now;
  const t = setTimeout(() => {
    dailyCheck();
    setInterval(dailyCheck, 24 * 60 * 60 * 1000);
  }, msAte);

  return () => clearTimeout(t);
}, [loaded]);

function addProduct(prod) {
  setProducts((ps) => [prod, ...ps]);
}

function removeProduct(id) {
  setProducts((ps) => ps.filter((p) => p.id !== id));
}

function dismissAlert(i) {
  setAlerts((a) => a.filter((_, idx) => idx !== i));
}

function clearAllAlerts() {
  setAlerts([]);
  setProducts((ps) => ps.map((p) => ({ ...p, hasAlert: false })));
}

```

```

const pendingCheck = products.filter((p) => shouldCheckToday(p)).length;

if (!loaded) {
  return (
    <div style={{ display: "flex", alignItems: "center", justifyContent: "center" }}>
      <div style={{ fontSize: "2rem", animation: "spin 1s linear infinite" }}>e</div>
      <div>Carregando Gingado Store...</div>
      <style>{@keyframes spin{to{transform:rotate(360deg)}}}</style>
    </div>
  );
}

return (
  <>
    <style>{`
      @import url('https://fonts.googleapis.com/css2?family=Bebas+Neue&family=O
      *,*::before,*::after{box-sizing:border-box;margin:0;padding:0}
      body{background:#0a0a0f;color:#e8e4dc;font-family:'Outfit',sans-serif}
      .app{min-height:100vh;background:radial-gradient(ellipse at 20% 0%,#1a0a2
      .header{background:linear-gradient(135deg,#1a0040 0%,#0d001f 100%);border
      .brand-logo{font-family:'Bebas Neue',sans-serif;font-size:1.9rem;color:#f
      .brand-sub{font-size:0.7rem;color:#a08050;letter-spacing:2px;text-transfo
      .header-actions{display:flex;gap:10px;align-items:center;flex-wrap:wrap}

      .btn-primary{background:#f5e642;color:#0a0a0f;border:none;padding:9px 20p
      .btn-primary:hover{background:#ffe000;transform:translateY(-1px);box-shad
      .btn-secondary{background:transparent;color:#a08050;border:1px solid #3a2
      .btn-secondary:hover{border-color:#f5e642;color:#f5e642}
      .btn-secondary:disabled{opacity:.4;cursor:not-allowed}

      .stats-bar{display:flex;gap:16px;padding:12px 24px;border-bottom:1px soli
      .stat-pill{display:flex;align-items:center;gap:6px;color:#6a5a4a}
      .stat-pill span{color:#e8e4dc;font-weight:600}
      .checking-badge{background:#1a1500;border:1px solid #3a3000;color:#f5e642
      @keyframes pulse{0%,100%{opacity:1}50%{opacity:0.6}}
      .pending-badge{background:#1a0040;border:1px solid #3a1a6a;color:#9a7ae0;

      .alert-stack{padding:14px 24px 0;display:flex;flex-direction:column;gap:1
      .alert-header-row{display:flex;justify-content:space-between;align-items:
      .alert-title{font-size:0.82rem;font-weight:700;color:#f5e642}
      .clear-all-btn{background:none;border:1px solid #3a2a0a;color:#6a5a30;fon
      .clear-all-btn:hover{border-color:#f5e642;color:#f5e642}
      .alert-card{background:#1a0a0a;border-left:4px solid #f44336;border-radiu
      .alert-card.price{border-color:#f5e642;background:#1a1500}
      .alert-card.stock{border-color:#2196f3;background:#001220}
    `}
  </style>
  </>
)

```

```

.alert-icon{font-size:1.4rem;flex-shrink:0}
.alert-body{flex:1}
.alert-body strong{font-size:0.88rem;color:#f5e642;display:block}
.alert-body p{font-size:0.8rem;color:#c0a870;margin-top:2px}
.alert-body small{font-size:0.7rem;color:#6a5a30}
.shopify-badge{background:#96bf48;color:#fff;font-size:0.7rem;font-weight
.dismiss-btn{background:none;border:none;color:#6a5a30;font-size:1rem;cur
.dismiss-btn:hover{color:#f5e642}

.main{padding:22px 24px}
.section-title{font-family:'Bebas Neue',sans-serif;font-size:1.1rem;lette
.section-title::after{content:'';flex:1;height:1px;background:#2a1a0a}

.empty{text-align:center;padding:80px 20px;color:#4a3a1a}
.empty-icon{font-size:4rem;margin-bottom:16px}
.empty h2{font-family:'Bebas Neue',sans-serif;font-size:1.8rem;color:#6a5
.empty p{font-size:0.88rem;margin-top:8px;color:#5a4a2a}

.product-grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(
.product-card{background:#12121a;border:1px solid #2a1a4a;border-radius:1
.product-card:hover{border-color:#5a3a8a;box-shadow:0 4px 24px #3a1a6a20}
.product-card.has-alert{border-color:#f5e642;box-shadow:0 0 20px #f5e6422
.alert-ribbon{background:#1a1500;border-bottom:1px solid #3a3000;color:#f

.card-header{display:flex;align-items:flex-start;gap:12px;padding:14px 14
.product-meta{flex:1;min-width:0}
.product-name{font-size:0.88rem;font-weight:600;color:#e8e4dc;line-height
.ml-link{font-size:0.73rem;color:#3483fa;text-decoration:none;transition:
.ml-link:hover{color:#6baaff}
.card-actions{display:flex;gap:6px;flex-shrink:0}
.refresh-btn{background:#1e1e30;border:1px solid #3a2a6a;color:#9a7ae0;wi
.refresh-btn:hover{background:#2a2040;border-color:#7a5ae0;color:#c0a0ff}
.refresh-btn.spinning{animation:spin .8s linear infinite}
.remove-btn{background:#200a0a;border:1px solid #5a1a1a;color:#8a4a4a;wid
.remove-btn:hover{background:#3a1010;color:#f44336;border-color:#f44336}

.card-body{padding:12px 14px}
.stat-row{display:grid;grid-template-columns:1fr 1fr 1fr auto;gap:10px;ma
.stat{display:flex;flex-direction:column;gap:3px}
.stat-label{font-size:0.65rem;color:#6a5a30;letter-spacing:0.5px;text-tra
.stat-val{font-size:0.9rem;font-weight:600;color:#e8e4dc}
.stat-val.price{color:#f5e642;font-size:1rem}
.stat-val.shopify-price{color:#96bf48}
.stat-badge{font-size:0.65rem;font-weight:700;padding:3px 8px;border-radi
.seller-row{display:flex;gap:14px;font-size:0.73rem;color:#7a6a4a;margin-
.changes-log{background:#1a1500;border:1px solid #3a3000;border-radius:8p
.changes-log strong{width:100%}

```

```

.change-tag{background:#2a2000;border:1px solid #4a3a00;border-radius:5px
.card-footer{display:flex;justify-content:space-between;font-size:0.68rem
.checked-today{color:#00c853}

.modal-overlay{position:fixed;inset:0;background:#00000090;display:flex;a
.modal{background:#12121a;border:1px solid #3a2a6a;border-radius:16px;pad
.modal h2{font-family:'Bebas Neue',sans-serif;font-size:1.4rem;letter-spa
.modal label{font-size:0.75rem;color:#8a7a5a;letter-spacing:0.5px;margin-
.modal-input{width:100%;background:#0e0e18;border:1px solid #3a2a6a;borde
.modal-input:focus{border-color:#7a5ae0}
.modal-input::placeholder{color:#4a3a2a}
.modal-hint{font-size:0.72rem;color:#5a4a2a;background:#1a1500;border-rad
.modal-btns{display:flex;gap:10px;margin-top:6px}
.btn-cancel{flex:1;background:#1e1e30;border:1px solid #3a2a6a;color:#8a7
.btn-cancel:hover{border-color:#5a4a8a;color:#c0b0e0}
.btn-add{flex:2;background:#f5e642;color:#0a0a0f;border:none;padding:10px
.btn-add:hover:not(:disabled){background:#ffe000}
.btn-add:disabled{opacity:.5;cursor:not-allowed}
.err-msg{color:#f44336;font-size:0.78rem;margin-bottom:10px;background:#1

@keyframes spin{to{transform:rotate(360deg)}}
@keyframes slideIn{from{transform:translateX(-20px);opacity:0}to{transfor
@keyframes fadeUp{from{transform:translateY(20px);opacity:0}to{transform:
`}</style>

<div className="app">
  <header className="header">
    <div>
      <div className="brand-logo">Gingado Store</div>
      <div className="brand-sub">Monitor de Fornecedores ML + Shopify</div>
    </div>
    <div className="header-actions">
      <button
        className="btn-secondary"
        onClick={async () => {
          setGlobalChecking(true);
          for (const p of productsRef.current) await checkProduct(p.id, tru
          setGlobalChecking(false);
        }}
        disabled={globalChecking || !products.length}
      >
        {globalChecking ? "Verificando..." : "🔍 Verificar Todos Agora"}
      </button>
      <button className="btn-primary" onClick={() => setShowModal(true)}>+
    </div>
  </header>

```

```

<div className="stats-bar">
  <div className="stat-pill">📦 Monitorados: <span>{products.length}</span>
  <div className="stat-pill">🔔 Alertas: <span>{alerts.length}</span></div>
  <div className="stat-pill">💾 Salvo na nuvem</div>
  {globalChecking && (
    <div className="checking-badge">🔄 Verificando produtos...</div>
  )}
  {!globalChecking && pendingCheck > 0 && (
    <div className="pending-badge">⌚ {pendingCheck} produto{pendingCheck}
  )}
  {!globalChecking && pendingCheck === 0 && products.length > 0 && (
    <div className="stat-pill">✅ Todos verificados hoje</div>
  )}
</div>

<AlertBanner alerts={alerts} onDismiss={dismissAlert} onClearAll={clearAl

<main className="main">
  {products.length === 0 ? (
    <div className="empty">
      <div className="empty-icon">📺</div>
      <h2>Nenhum produto monitorado</h2>
      <p>Adicione o link de um fornecedor do Mercado Livre para começar.</p>
      <p style={{ marginTop: 8 }}>Os dados ficam salvos na nuvem – nunca
    </div>
  ) : (
    <>
      <div className="section-title">Produtos Monitorados</div>
      <div className="product-grid">
        {products.map((p) => (
          <ProductCard
            key={p.id}
            product={p}
            onRefresh={(id) => checkProduct(id, true)}
            onRemove={removeProduct}
            isChecking={!!checking[p.id]}
          />
        ))}
      </div>
    </>
  )}
</main>
</div>

{showModal && <AddModal onAdd={addProduct} onClose={() => setShowModal(false)}
</>
);

```

